Architecture

The framework separates **generic infrastructure** from **application-specific code**, so adopting it for a different product means replacing one layer, not rewriting everything.

```
tests/                Specs: business scenarios only, no locators, no HTTP details
  |
src/fixtures/         Dependency injection: wires page objects, API clients and auth into
tests
  |
  +-- src/pages/      Page objects (application-specific)
  +-- src/api/        API clients (base class is generic; domain clients are application-
specific)
  +-- src/data/       Test data builders and sample values (application-specific)
  +-- src/db/         Read-only database queries for integrity checks (application-specific)
  |
src/config/           Environment profiles and credential resolution (generic)
src/utils/            Tags, requirement links, recording stamp and shared helpers (generic)

tools/                Node tools run by npm scripts and CI: start the app, reports,
traceability, evidence pack
config/               Report settings (Allure)
reports/              Everything a run generates: _raw, standard (default), regulated (on
request)
```

Generic layer (reusable as-is)

- `src/config/env.ts` : selects a profile with `TEST_ENV` , reads `.env.<profile>` , lets real environment variables override. Credentials resolve lazily, per role.
- `src/api/base-api-client.ts` : typed request helper with explicit status expectations and readable failures.
- `src/fixtures/base.fixture.ts` : base `test` object that all specs import.
- `src/utils/tags.ts` : central tag names for selecting suites.
- `src/utils/requirements.ts` : `requirement('REQ-...')` turns requirement IDs into tags, the link behind the traceability matrix.
- `src/utils/evidence-stamp.ts` : the live date-and-time bar used in capture mode.
- `tools/` : plain Node scripts. `start-app.mjs` is application-specific; the rest (clean, traceability, validation report, evidence pack) are generic.

Application layer (replace per product)

- Page objects, domain API clients, test data, and the specs themselves.

Design patterns used

Pattern	Where	Why
Page Object Model	`src/pages/`	Locators live in one class per page; tests call business actions.
Component Object	`src/pages/components/header.ts`	Shared UI (the header) is composed into pages, not copied.
Fixture-based dependency inject	`src/fixtures/`	Tests request what they need; setup and cleanup are automatic.
Service Object (API client)	`src/api/`	HTTP is wrapped in typed, business-level methods.
Template Method	`BaseApiClient.send()`	Shared request, status check and error handling; subclasses add endpoints.
Layered architecture	tests, fixtures, pages and API, config	One-way dependencies make the application layer replaceable.
Environment-based configuration	`src/config/env.ts`	Profiles via `TEST_ENV`; nothing environment-specific in code.
Cross-cutting automatic fixture	`reportMetadata` in `base.fixture.ts`	Report labels are applied to every test without touching specs.
Test Data Builder	`src/data/builders.ts`	Every call returns unique, valid data, so parallel tests never collide.
Repository (read-only)	`src/db/toolshop-db.ts`	SQL lives in one class, so specs say what they check, not how to query it.
Self-cleaning fixture	`newBrand`, `newProduct`, `newUser`	Created records are removed after the test, even when it fails.
Known-issue (expected failure)	`Tags.knownIssue` with `test.fail`	Application defects are documented without leaving the build red.
Requirement tagging	`requirement()` in `src/utils`	A test names what it verifies; the matrix is built from the tags.
Two report profiles	`reports/standard`, `reports/regulated`	Everyday reports stay light; validation evidence is built only on request.
Evidence pack	`tools/evidence-bundle.mjs`	One folder with a manifest and checksums shows a run was not changed.

Pattern	Where	Why
Variant config	`playwright.capture.config.ts`	Recording everything is a separate config, so defaults stay economical.

Reports

A run writes to `reports/_raw`. From there, `npm run report:allure` builds the **standard** report (the default) and `npm run report:validation` builds the **regulated** evidence pack (on request). They are kept in separate folders on purpose. See reporting.md.

Regulated runs

The optional regulated mode lives in `tools/` and `config/regulated.json`, and is off until it is switched on. `tools/regulated-run.mjs` runs the suite in capture mode and builds the evidence pack through `tools/evidence-bundle.mjs`. Small libraries in `tools/lib` do the work: `regulated-config` (the settings and the rules), `change-record` and `git` (what changed outside the run), `compare` (dry run against formal run), `archive` and `zip` (the sealed zip and the audit log) and `verify` (checking an archive). They have their own fast tests (`npm run test:tools`). See regulated/dry-run-and-formal-run.md.

Principles

- Tests state intent; page objects and clients own mechanics.
- Test state is created through the API, then verified in the UI where the UI is what is under test.
- Every test is independent and can run in parallel.
- No environment values or credentials in code.